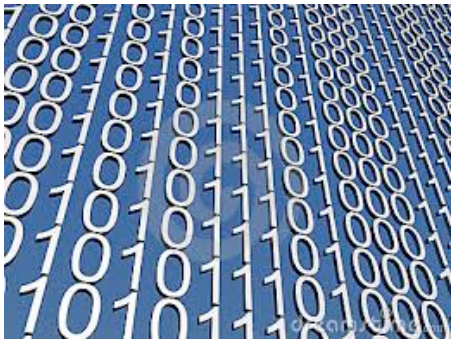


Number Systems and Number Representation



For Your Amusement



Question: Why do computer programmers confuse Christmas and Halloween?

Answer: Because 25 Dec = 31 Oct

-- http://www.electronicsweekly.com

Agenda



Number Systems

Finite representation of unsigned integers

Finite representation of signed integers

Finite representation of rational numbers

Radix r number systems

Binary Coded Decimal Code (BCD)

Specialized Systems

Excess-3

Gray and ASCII codes

Parity codes

The Decimal Number System



Name

- "decem" (Latin) => ten

Characteristics

- Ten symbols
  - **0 1 2 3 4 5 6 7 8 9**
- Positional
  - **2945 ≠ 2495**
  - **2945 = (2*10^3) + (9*10^2) + (4*10^1) + (5*10^0)**

(Most) people use the decimal number system



The Binary Number System



Name

- "binarius" (Latin) => two

Characteristics

- Two symbols
  - `0 1`
- Positional
  - $1010_B \neq 1100_B$

Most (digital) computers use the binary number system



Terminology

- **Bit**: a binary digit
- **Byte**: (typically) 8 bits

Decimal-Binary Equivalence



Decimal	Binary
0	0
1	1
2	10
3	11
4	100
5	101
6	110
7	111
8	1000
9	1001
10	1010
11	1011
12	1100
13	1101
14	1110
15	1111

Decimal	Binary
16	10000
17	10001
18	10010
19	10011
20	10100
21	10101
22	10110
23	10111
24	11000
25	11001
26	11010
27	11011
28	11100
29	11101
30	11110
31	11111
...	...

Decimal-Binary Conversion



Binary to decimal: expand using positional notation

$$
\begin{aligned}
100101_B &= (1*2^5)+(0*2^4)+(0*2^3)+(1*2^2)+(0*2^1)+(1*2^0) \\
&= 32 + 0 + 0 + 4 + 0 + 1 \\
&= 37
\end{aligned}
$$

Decimal-Binary Conversion



Decimal to binary: do the reverse

- Determine largest power of 2 ≤ number; write template

$$37 = (?*2^5)+(?*2^4)+(?*2^3)+(?*2^2)+(?*2^1)+(?*2^0)$$

- Fill in template

$$37 = (1*2^5)+(0*2^4)+(0*2^3)+(1*2^2)+(0*2^1)+(1*2^0)$$

$$\begin{array}{r} 37 \\ -32 \\ \hline 5 \\ -4 \\ \hline 1 \\ -1 \\ \hline 0 \end{array}$$

$$100101_B$$

Decimal-Binary Conversion



Decimal to binary shortcut

- Repeatedly divide by 2, consider remainder

```
37 / 2 = 18 R 1
18 / 2 =  9 R 0
 9 / 2 =  4 R 1
 4 / 2 =  2 R 0
 2 / 2 =  1 R 0
 1 / 2 =  0 R 1
```



Read from bottom to top: 100101_B

The Hexadecimal Number System



Name

- "hexa" (Greek) => six
- "decem" (Latin) => ten

Characteristics

- Sixteen symbols
  - **0 1 2 3 4 5 6 7 8 9 A B C D E F**
- Positional
  - $\mathbf{A13D_H \neq 3DA1_H}$

Computer programmers often use the hexadecimal number system



Decimal-Hexadecimal Equivalence



Decimal	Hex
0	0
1	1
2	2
3	3
4	4
5	5
6	6
7	7
8	8
9	9
10	A
11	B
12	C
13	D
14	E
15	F

Decimal	Hex
16	10
17	11
18	12
19	13
20	14
21	15
22	16
23	17
24	18
25	19
26	1A
27	1B
28	1C
29	1D
30	1E
31	1F

Decimal	Hex
32	20
33	21
34	22
35	23
36	24
37	25
38	26
39	27
40	28
41	29
42	2A
43	2B
44	2C
45	2D
46	2E
47	2F
...	...

Decimal-Hexadecimal Conversion



Hexadecimal to decimal: expand using positional notation

$$25_H = (2*16^1) + (5*16^0)$$
$$= 32 + 5$$
$$= 37$$

Decimal to hexadecimal: use the shortcut

```
37 / 16 = 2 R 5
 2 / 16 = 0 R 2
```



Read from bottom to top: 25_H

Binary-Hexadecimal Conversion



Observation: $16^1 = 2^4$

- Every 1 hexadecimal digit corresponds to 4 binary digits

Binary to hexadecimal

$$1010000100111101_B$$
$$A \quad 1 \quad 3 \quad D_H$$

Digit count in binary number not a multiple of 4 => pad with zeros on left

Hexadecimal to binary

$$A \quad 1 \quad 3 \quad D_H$$
$$1010000100111101_B$$

Discard leading zeros from binary number if appropriate

Is it clear why programmers often use hexadecimal?

The Octal Number System



Name

- "octo" (Latin) => eight

Characteristics

- Eight symbols
  - `0 1 2 3 4 5 6 7`
- Positional
  - $1743_o \neq 7314_o$

Computer programmers often use the octal number system



Decimal-Octal Equivalence



Decimal	Octal
0	0
1	1
2	2
3	3
4	4
5	5
6	6
7	7
8	10
9	11
10	12
11	13
12	14
13	15
14	16
15	17

Decimal	Octal
16	20
17	21
18	22
19	23
20	24
21	25
22	26
23	27
24	30
25	31
26	32
27	33
28	34
29	35
30	36
31	37

Decimal	Octal
32	40
33	41
34	42
35	43
36	44
37	45
38	46
39	47
40	50
41	51
42	52
43	53
44	54
45	55
46	56
47	57
...	...

Decimal-Octal Conversion



Octal to decimal: expand using positional notation

$$37_O = (3*8^1) + (7*8^0)$$
$$= 24 + 7$$
$$= 31$$

Decimal to octal: use the shortcut

```
31 / 8 = 3 R 7
 3 / 8 = 0 R 3
```



Read from bottom to top: 37_O

Binary-Octal Conversion



Observation: $8^1 = 2^3$

- Every 1 octal digit corresponds to 3 binary digits

Binary to octal

001010000100111101_B
$1\ \ 2\ \ 0\ \ 4\ \ 7\ \ 5_O$

Digit count in binary number not a multiple of 3 => pad with zeros on left

Octal to binary

$1\ \ 2\ \ 0\ \ 4\ \ 7\ \ 5_O$
001010000100111101_B

Discard leading zeros from binary number if appropriate

Is it clear why programmers sometimes use octal?

Unsigned Data Types: Java vs. C



Java has type

- **`int`**
  - Can represent signed integers

C has type:

- **`signed int`**
  - Can represent signed integers
- **`int`**
  - Same as **`signed int`**
- **`unsigned int`**
  - Can represent only unsigned integers

To understand C, must consider representation of both unsigned and signed integers

Representing Unsigned Integers



Mathematics

- Range is 0 to ∞

Computer programming

- Range limited by computer's **word** size
- Word size is n bits => range is 0 to $2^n - 1$
- Exceed range => **overflow**

~~Nobel~~ computers with gcc217

- n = 32, so range is 0 to $2^{32} - 1$ (4,294,967,295)

Pretend computer

- n = 4, so range is 0 to $2^4 - 1$ (15)

Hereafter, assume word size = 4

- All points generalize to word size = 32, word size = n

Representing Unsigned Integers



On pretend computer

Unsigned Integer	Rep
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001
10	1010
11	1011
12	1100
13	1101
14	1110
15	1111

Adding Unsigned Integers



Addition

```
          1
   3      0011B
+ 10    + 1010B
  --      ----
  13      1101B
```

Start at right column
Proceed leftward
Carry 1 when necessary

```
         11
   7      0111B
+ 10    + 1010B
  --      ----
   1     10001B
```

Beware of overflow

How would you detect overflow programmatically?

Results are mod 2^4

Subtracting Unsigned Integers



Subtraction

```
          12
         0202
  10      1010B
-  7    - 0111B
  --      ----
   3      0011B
```

Start at right column
Proceed leftward
Borrow 2 when necessary

```
          2
   3      0011B
- 10    - 1010B
  --      ----
   9      1001B
```

Beware of overflow

How would you detect overflow programmatically?

Results are mod 2^4

When subtracting unsigned integers, an " underflow" occurs if the result would be a negative number.

Signed Magnitude



Integer	Rep
-7	1111
-6	1110
-5	1101
-4	1100
-3	1011
-2	1010
-1	1001
-0	1000
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111

Definition

High-order bit indicates sign

0 => positive

1 => negative

Remaining bits indicate magnitude

$1101_B = -101_B = -5$

$0101_B = 101_B = 5$

Signed Magnitude (cont.)



Integer	Rep
-7	1111
-6	1110
-5	1101
-4	1100
-3	1011
-2	1010
-1	1001
-0	1000
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111

Computing negative

neg(x) = flip high order bit of x

$\mathbf{neg}(0101_B) = 1101_B$

$\mathbf{neg}(1101_B) = 0101_B$

Pros and cons

+ easy for people to understand
+ symmetric
- two reps of zero

Ones' Complement



Integer	Rep
-7	1000
-6	1001
-5	1010
-4	1011
-3	1100
-2	1101
-1	1110
-0	1111
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111

Definition

High-order bit has weight -7

1010_B = (1*-7)+(0*4)+(1*2)+(0*1)
= -5

0010_B = (0*-7)+(0*4)+(1*2)+(0*1)
= 2

Ones' Complement (cont.)



Integer	Rep
-7	1000
-6	1001
-5	1010
-4	1011
-3	1100
-2	1101
-1	1110
-0	1111
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111

Computing negative

neg(x) = ~x

$neg(0101_B) = 1010_B$

$neg(1010_B) = 0101_B$

Computing negative (alternative)

neg(x) = 1111_B - x

$neg(0101_B) = 1111_B - 0101_B$

$= 1010_B$

$neg(1010_B) = 1111_B - 1010_B$

$= 0101_B$

Pros and cons

+ symmetric

- two reps of zero

Similar pros and cons to sign-magnitude

Two's Complement



Integer	Rep
-8	1000
-7	1001
-6	1010
-5	1011
-4	1100
-3	1101
-2	1110
-1	1111
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111

Definition

High-order bit has weight -8

1010_B = (1*-8)+(0*4)+(1*2)+(0*1)
= -6

0010_B = (0*-8)+(0*4)+(1*2)+(0*1)
= 2

Two's Complement (cont.)



Integer	Rep
-8	1000
-7	1001
-6	1010
-5	1011
-4	1100
-3	1101
-2	1110
-1	1111
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111

Computing negative

neg(x) = ~x + 1

neg(x) = onescomp(x) + 1

$neg(0101_B) = 1010_B + 1 = 1011_B$

$neg(1011_B) = 0100_B + 1 = 0101_B$

Pros and cons

- not symmetric ("extra" negative number)
+ one representation of zero

Two's Complement (cont.)



Almost all computers use two's complement to represent signed integers

Why?

- Arithmetic is easy
  - Will become clear soon

Hereafter, assume two's complement representation of signed integers

Adding Signed Integers



pos + pos

```
          11
   3      0011B
 + 3    + 0011B
  --      ----
   6      0110B
```

pos + pos (overflow)

```
          111
   7      0111B
 + 1    + 0001B
  --      ----
  -8      1000B
```

pos + neg

```
         1111
   3      0011B
+ -1    + 1111B
  --      ----
   2     10010B
```

How would you detect overflow programmatically?

neg + neg

```
          11
  -3      1101B
+ -2    + 1110B
  --      ----
  -5     11011B
```

neg + neg (overflow)

```
         1 1
  -6      1010B
+ -5    + 1011B
  --      ----
   5     10101B
```

Adding Signed Integers



The key rule for two's complement overflow:

- If you add two numbers with the **same sign** (both positive or both negative) and the **result has the opposite sign, an overflow has occurred**.
- If you add two numbers with **different signs**, overflow is **impossible.** The sum of a positive and a negative number will always be closer to zero than one of the operands, so it will fit within the range.

Subtracting Signed Integers



Perform subtraction with borrows

or

Compute two's comp and add

```
          1
          22
  3       0011B
- 4     - 0100B
 --       ----
 -1       1111B
```

```
   3       0011B
+ -4     + 1100B
  --       ----
  -1       1111B
```

```
 -5       1011B
- 2     - 0010B
 --       ----
 -7       1001B
```

```
          111
  -5       1011
+ -2     + 1110
  --       ----
  -7      11001
```

Parts of a Fractional Number

The meaning of the "decimal point" is as follows:

$$\begin{aligned} 123.406_{10} =& 1 \times 10^2 + 2 \times 10^1 + 3 \times 10^0 + \\ & 4 \times 10^{-1} + 0 \times 10^{-2} + 6 \times 10^{-3} \\ =& 123.406_{10} \end{aligned} \qquad \begin{aligned} 123 &= 100 + 20 + 3 \\ 0.406 &= \frac{4}{10} + \frac{6}{1000} \end{aligned}$$

Changing to base 2 induces a "binary point" with similar meaning:

$$\begin{aligned} 110.101_2 =& 1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 + \\ & 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3} \\ =& 6.625_{10} \end{aligned} \qquad \begin{aligned} 6 &= 4 + 2 \\ 0.625 &= \frac{1}{2} + \frac{1}{8} \end{aligned}$$

One *could* represent fractional numbers with a **fixed point** e.g.

- 32 bit fractional number with
- 10 bits left of Binary Point (integer part)
- 22 bits right of Binary Point (fractional part)

BUT most applications require a more flexible scheme

Scientific Notation for Numbers

"Scientific" or "Engineering" notation for numbers with a fractional part is

Standard	Scientific	`printf("%.4e",x);`
123.456	1.23456×10^{2}	`1.2346e+02`
50.01	5.001×10^{1}	`5.0010e+01`
3.14159	3.14159×10^{0}	`3.1416e+00`
0.54321	5.4321×10^{-1}	`5.4321e-01`
0.00789	7.89×10^{-3}	`7.8900e-03`

- **Always** includes one **non-zero** digit left of decimal place
- Has some **significant** digits after the decimal place
- Multiplies by a **power of 10** to get actual number

Binary Floating Point Layout Uses Scientific Convention

- Some bits for integer/fractional part
- Some bits for exponent part
- All in base 2: 1's and 0's, powers of 2

Conversion Example

Below steps convert a decimal number to a fractional binary number equivalent then adjusts to scientific representation.

```
float fl = -248.75;

                  7  6  5  4 3 2 1 0   -1  -2
-248.75 = -(128+64+32+16+8+0+0+0).(1/2+1/4)
        = -11111000.11 *2^0
           76543210 12
        = -1111100.011 *2^1
           6543210 123
        = -111110.0011 *2^2
           543210 1234
           ...
             MANTISSA    EXPONENT
        = -1.111100011 * 2^7
           0 123456789
```

Mantissa ≡ Significand ≡ Fractional Part

Read More: https://vishalchovatiya.com/posts/how-floating-point-no-is-stored-memory/

IEEE Floating Point Representation



Common finite representation: **IEEE floating point**

- More precisely: ISO/IEEE 754 standard

Using 32 bits (type float in C):

- 1 bit: sign (0=>positive, 1=>negative)
- 8 bits: exponent + 127
- 23 bits: binary fraction of the form 1.*ddddddddddddddddddddddd*

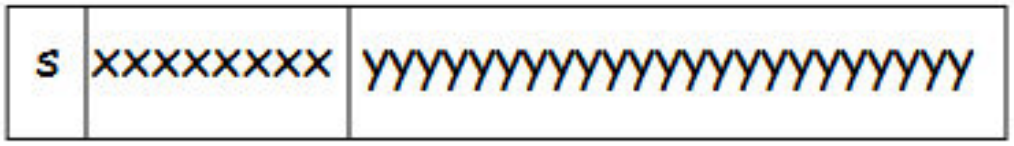
Using 64 bits (type double in C):

- 1 bit: sign (0=>positive, 1=>negative)
- 11 bits: exponent + 1023
- 52 bits: binary fraction of the form 1.*ddddddddddddddddddddddd

IEEE Floating Point Representation



IEEE 754 single-precision (32 bits)



Single precision

1 8 23 bits

Largest = 1. 1 1 1 ... x 2^{+127} ≈ 2 x 10^{+38}

Smallest = 1.000 ... x 2^{-128} ≈ 1 x 10^{-38}

These can be positive and negative, depending on s.

(But there are exceptions too)

Floating Point Example



11000001110110110000000000000000

32-bit representation

Sign (1 bit):

- 1 => negative

Exponent (8 bits):

- $10000011_B = 131$
- $131 - 127 = 4$

Fraction (23 bits):

- $1.10110110000000000000000_B$
- $1 + (1*2^{-1})+(0*2^{-2})+(1*2^{-3})+(1*2^{-4})+(0*2^{-5})+(1*2^{-6})+(1*2^{-7})$
  $= 1.7109375$

Number:

- $-1.7109375 * 2^4 = -27.375$

Floating Point Example (2)

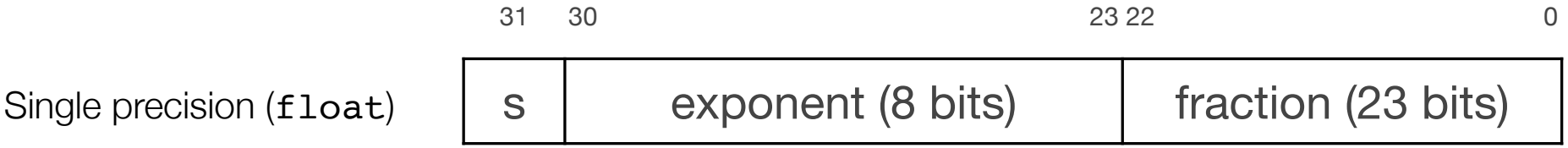


Example: Convert 0.625 to binary

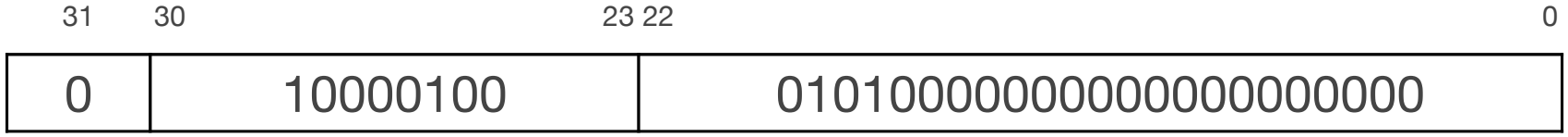
- 0.625 * 2 = 1.25 → integer part is 1. New fraction is 0.25
- 0.25 * 2 = 0.5 → integer part is 0. New fraction is 0.5
- 0.5 * 2 = 1.0 → integer part is 1. New fraction is 0.0. Stop

Reading the integer parts from top to bottom gives you the binary fraction: 0.101

Let's try some more…



- Example:



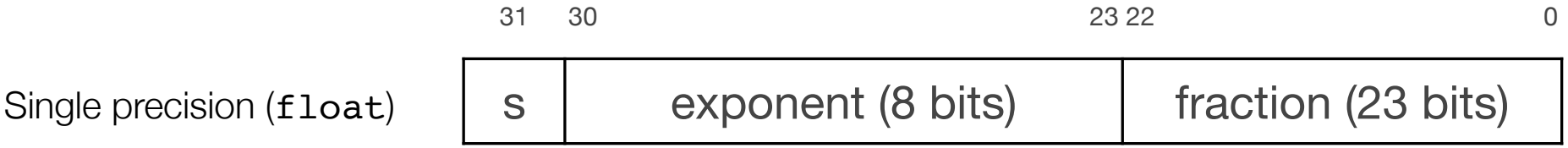
- Sign: 0 (positive)
- Exponent: 10000100 = 132 (biased), so exponent of 2 will be 132 - 127 = 5
- Fraction: 0101, which is assumed to be 1.0101 (binary), which is:

$$1 \times 2^0 + 0 \times 2^{-1} + 1 \times 2^{-2} + 0 \times 2^{-3} + 1 \times 2^{-4} = 1.3125$$

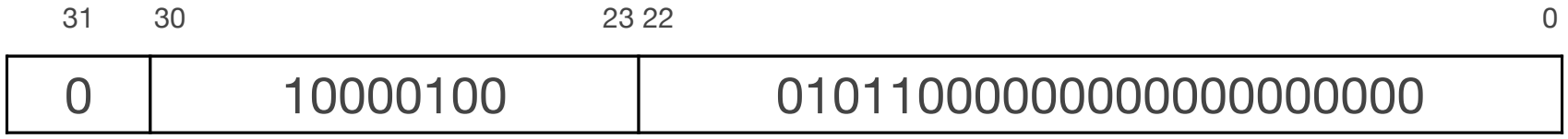
- Therefore, this number represents +1.3125 x 2^5 = 42.0 **(to the converter!)**

https://www.h-schmidt.net/FloatConverter/IEEE754.html

Let's try some more…



- Example:

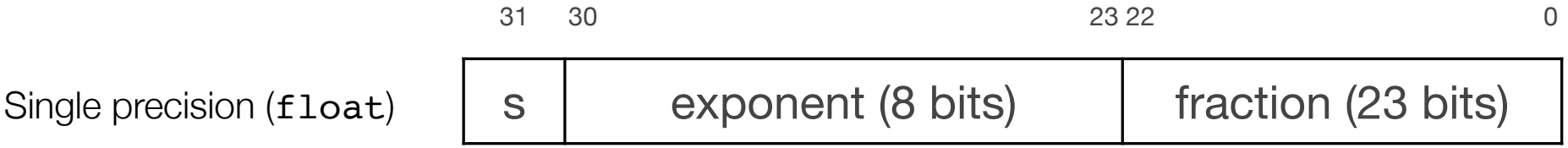


- Sign: 0 (positive)
- Exponent: 10000100 = 132 (biased), so exponent of 2 will be 132 - 127 = 5
- Fraction: 01011, which is assumed to be 1.01011 (binary), which is:

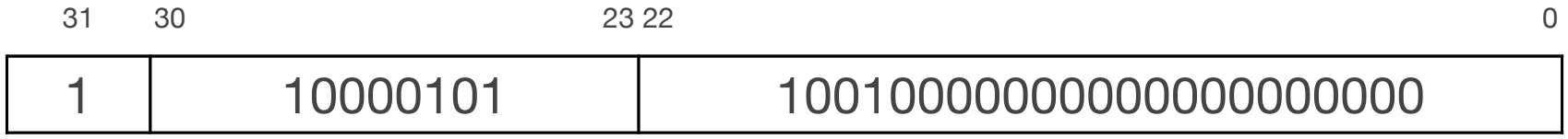
$$1 \times 2^0 + 0 \times 2^{-1} + 1 \times 2^{-2} + 0 \times 2^{-3} + 1 \times 2^{-4} + 1 \times 2^{-5} = 1.34375$$

- Therefore, this number represents +1.34375 x 2^5 = 43.0 **(to the converter!)**

Let's try some more…



- Example:



- Sign: 1 (negative)
- Exponent: 10000101 = 133 (biased), so exponent of 2 will be 133 - 127 = 6
- Fraction: 1001, which is assumed to be 1.1001 (binary), which is:

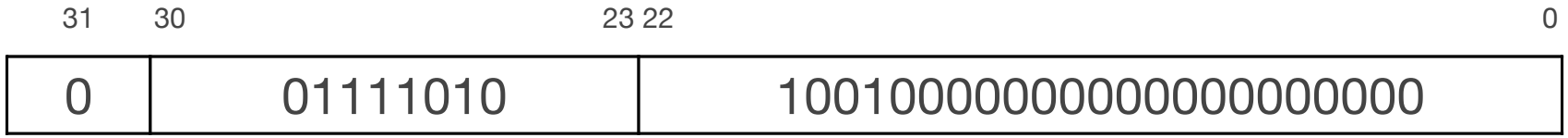
$$1 \times 2^0 + 1 \times 2^{-1} + 0 \times 2^{-2} + 0 \times 2^{-3} + 1 \times 2^{-4} = 1.5625$$

- Therefore, this number represents -1.5625 x 2^6 = -100.0 **(to the converter!)**

Let's try some more…



- Example:



- Sign: 0 (positive)
- Exponent: 01111010 = 122 (biased), so exponent of 2 will be 122 - 127 = -5
- Fraction: 1001, which is assumed to be 1.1001 (binary), which is:

$$1 \times 2^0 + 1 \times 2^{-1} + 0 \times 2^{-2} + 0 \times 2^{-3} + 1 \times 2^{-4} = 1.5625$$

- Therefore, this number represents +1.5625 x 2^{-5} = 0.048828125

(to the converter!)

Floating Point Consequences



"Machine epsilon": smallest positive number you can add to 1.0 and get something other than 1.0

For float: $\varepsilon \approx 10^{-7}$

- No such number as 1.000000001
- Rule of thumb: "almost 7 digits of precision"

For double: $\varepsilon \approx 2 \times 10^{-16}$

- Rule of thumb: "not quite 16 digits of precision"

These are all *relative* numbers

Floating Point Consequences, cont



Just as decimal number system can represent only some rational numbers with finite digit count…

- Example: 1/3 ***cannot*** be represented

Decimal Approx	Rational Value
.3	3/10
.33	33/100
.333	333/1000
...	

Binary number system can represent only some rational numbers with finite digit count

- Example: 1/5 ***cannot*** be represented

Binary Approx	Rational Value
0.0	0/2
0.01	1/4
0.010	2/8
0.0011	3/16
0.00110	6/32
0.001101	13/64
0.0011010	26/128
0.00110011	51/256
...	

Beware of **roundoff error**

- Error resulting from inexact representation
- Can accumulate
- Be careful when comparing two floating-point numbers for equality

Radix *r* Number System

- Let $N)_r$ be a radix *r* number in a positional weighting number system, then

$$N)_r = d_{n-1}\, r^{n-1} + d_{n-2}\, r^{n-2} + \ldots + d_1\, r^1 + d_0\, r^0$$

$$. \; d_{-1}\, r^{-1} + d_{-2}\, r^{-2} + \ldots\ldots + d_{-m}\, r^{-m}$$

Radix point

Where: *r* = radix , d_i = digit at position *i* ,
r^i = weight of position *i* ,
n = number of integral digits in *N* ,
m = number of fractional digits in *N*

Radix *r* Number System

- **Binary** *r* = 2, digits are 0; 1

$$(101.01)_2 = 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 \bullet 0 \cdot 2^{-1} + 1 \cdot 2^{-2}$$

- **Octal** *r* = 8, digits are 0; 1; 2; 3; 4; 5; 6; 7

$$(62.357)_8 = 6 \cdot 8^1 + 2 \cdot 8^0 \bullet 3 \cdot 8^{-1} + 5 \cdot 8^{-2} + 7 \cdot 8^{-3}$$

- **Decimal** *r* = 10, digits are 0; 1; 2; 3; 4; 5; 6; 7; 8; 9

$$(468.93)_{10} = 4 \cdot 10^2 + 6 \cdot 10^1 + 8 \cdot 10^0 \bullet 9 \cdot 10^{-1} + 3 \cdot 10^{-2}$$

Radix *r* Number System

- **Hexadecimal** *r* = 16, digits are 0*;* 1*;* 2*;* 3*;* 4*;* 5*;* 6*;* 7*;* 8*;* 9*;* A,B,C,D,E,F

$$(1C.A0F)_{16} = 1\cdot 16^{1} + C\cdot 16^{0} \bullet A\cdot 16^{-1} + 0\cdot 16^{-2} + F\cdot 16^{-3}$$

- *r* = 5 ?? digits are 0; 1; 2; 3; 4
- *r = 12 ?? Digits are 0; 1; …..; A ; B*

Binary Coded Decimal Code (BCD)

- Group of 4 bits represents on decimal digit.
- $(18)_{10} = (0001\ 1000)_{BCD}$
- The binary combinations 1010 through 1111 are not used and have no meaning in BCD.

Table 1.4
Binary-Coded Decimal (BCD)

Decimal Symbol	BCD Digit
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

BCD Addition

- BCD addition is similar to unsigned 4-bit binary addition, except that a correction is required if the result exceeds $1001)_2 = 9)_{10}$
- The correct result is obtained by adding $0110)_2 = 6)_{10}$
- Perform in BCD the addition 6+8

$$\begin{array}{rl} & 0110 \\ + & 1000 \\ \hline & 1110 \end{array}$$

Correction needed

$$\begin{array}{rl} + & 1110 \\ & 0110 \\ \hline & 1\ 0100 \\ & 1 \quad\ 4 \end{array}$$

Result is 14

BCD Addition

- Perform in BCD the addition 3+3

$$\begin{array}{r} 0011 \\ +\ 0011 \\ \hline 0110 \end{array}$$

No correction needed

Result is 6

Excess-3

- For the excess-3 code the code word for each decimal digit is the corresponding BCD code word plus $0011)_2 = 3)_{10}$

Decimal number	2 ↓	3	4	5	
BCD Code	0010	0011	0100	0101	Ans.
	+ 0011	+ 0011	+ 0011	+ 0011	Add 3 (or 0011) in each BCD
Excess-3 code	0101	0110	0111	1000	Ans.

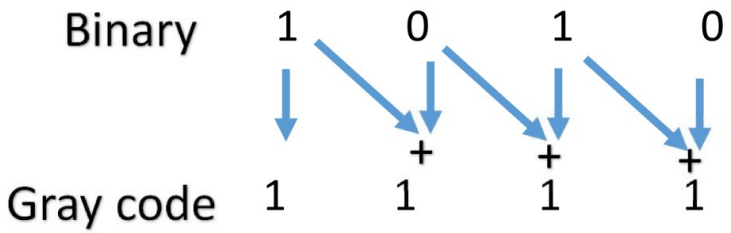
Decimal	*BCD*	*Excess-3*
0	0000	0011
1	0001	0100
2	0010	0101
3	0011	0110
4	0100	0111
5	0101	1000
6	0110	1001
7	0111	1010
8	1000	1011
9	1001	1100

Gray Code

is not an arithmetic code! Used for data storage and transmission

- The advantage of the Gray code over the straight binary number sequence is that only one bit in the code group changes in going from one number to the next.

Binary to Gray code Conversion



Gray code to Binary Conversion

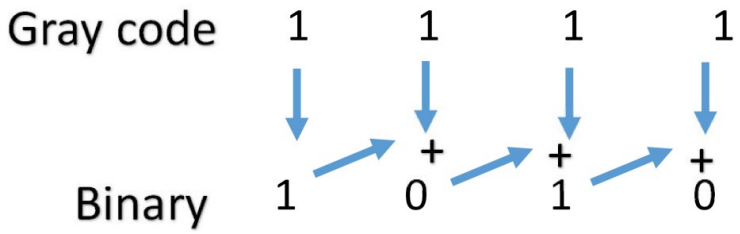


Table 1.6
Gray Code

Gray Code	Decimal Equivalent
0000	0
0001	1
0011	2
0010	3
0110	4
0111	5
0101	6
0100	7
1100	8
1101	9
1111	10
1110	11
1010	12
1011	13
1001	14
1000	15

ASCII Code

is not an arithmetic code! Used for data storage and transmission

- American Standard Code for Information Interchange
- ASCII is a 7-bit code, frequently used with an 8th bit for error detection (more about that in a bit)

Character	ASCII (bin)	ASCII (hex)	Decimal	Octal
A	1000001	41	65	101
B	1000010	42	66	102
C	1000011	43	67	103
…				
Z				
a				

Parity Codes

- Parity codes are formed by concatenating a *parity bit, P* to each code word of C
- In an *odd-parity code*, the parity bit is specified so that the total number of ones is odd
- In an even-*parity code*, the parity bit is specified so that the total number of ones is even



1 1 0 0 0 0 1 1

↑

Added even parity bit

0 1 0 0 0 0 1 1

↑

Added odd parity bit

Parity Code Example

- Concatenate a parity bit to the ASCII code for the characters 0, X, and = to produce both odd-parity and even-parity codes

Character	ASCII	Odd-Parity ASCII	Even-Parity ASCII
0	0110000	10110000	00110000
X	1011000	01011000	11011000
=	0111100	10111100	00111100